KIV/ZOS – Základy operačních systémů

Semestrální práce

PseudoFAT

Zjednodušený souborový systém založený na FAT

Jméno: Jan Kopačka
Osobní číslo: A24B0346P
Email: jankop@students.zcu.cz
Datum: 28. října 2026

Obsah

1 Úvod **3**
1.1 Účel projektu . . . 3
1.2 Adresářová struktura . . . 3
1.3 Architektura . . . 3

2 Architektonická rozhodnutí **5**
2.1 Návrh . . . 5
2.1.1 Rozložení disku . . . 5
2.1.2 FAT tabulka . . . 5
2.1.3 Limity velikostí . . . 6
2.1.4 Adresáře . . . 6
2.1.5 Překlad cesty . . . 7
2.1.6 Pořadí zápisů a operace . . . 7
2.1.7 check a bug . . . 8
2.1.8 load . . . 8
2.1.9 MiB a MB . . . 8
2.1.10 Multiplatformnost . . . 9
2.2 Implementace . . . 9
2.2.1 Příkazy jako `do_*` . . . 9
2.2.2 Umělé `Next` v `get_chain` . . . 9
2.2.3 `pub(crate)` v `shell.rs` . . . 9
2.2.4 Dlouhé názvy v `paths.rs` . . . 9
2.2.5 Další chytáky . . . 10

3 Největší problémy **11**

1 Úvod

1.1 Účel projektu

Program implementuje zjednodušený souborový systém založený na FAT. Je napsaný v Rustu, což bylo pro tuto práci povoleno i přes požadavky zadání. Celý systém je uložený v jednom souboru na hostitelském disku a jeho obsah zůstává zachovaný mezi spuštěními. Program se spouští s jedním parametrem, cestou k tomuto souboru:

```
pseudofat myfs.dat
```

Ovládá se příkazy ze zadání (`format`, `mkdir`, `incp`, `cat` apod.) zadávanými na standardní vstup. Podle loginu (`jankop`) jsou navíc implementované příkazy `check` (kontrola konzistence) a `bug` (záměrné poškození). Úplné znění zadání je v repozitáři v `docs/cz/Zadani2026.pdf`.

Výstupy `OK`, textové výstupy (`ls`, `pwd`, `cat`, `info`, `check`) a chyby parsování jdou na `stdout`. Chyby příkazů (`FILE NOT FOUND`, `PATH NOT FOUND`, `EXIST`, ...) jdou na `stderr`.

Sestavení a spuštění zajišťuje `Makefile`: `make build` (release binárka do `dist/`), `make run`, `make test`, `make clean`.

1.2 Adresářová struktura

```
Cargo.toml, Makefile
src/
  main.rs          vstupni bod, zpracovani argumentu
  shell.rs         smycka cteni prikazu, volani prikazu, vypis vysledku
  command.rs       parser prikazu (Command), Output, CommandExecError
  command/         jeden soubor = jeden prikaz (cat.rs, cp.rs, ..., load.rs)
  filesystem.rs    struktura FileSystem, FsError, operace se soubory
  filesystem/
    superblock.rs  superblok: rozlozeni disku, validace pri otevreni
    fat.rs         FAT tabulka: alokace, retezce, uvolnovani
    dir.rs         adresare a jejich polozky, jmena 8.3
    paths.rs       preklad cest
    consistency.rs logika pro prikazy check a bug
    disk.rs        nejnizsi vrstva: cteni/zapis bajtu na offsetu
  host.rs          cteni/zapis souboru na hostitelskem disku
  bytes.rs         serializace cisel (little-endian)
  config.rs        konstanty (velikost clusteru, limity)
  helpers.rs       pomocne funkce (varovani)
tests/
  integration.rs   integracni testy (spousti binarku, kontroluje vystup)
docs/cz/           tato dokumentace
```

1.3 Architektura

Program je rozdělený do vrstev. Každá vrstva volá jen vrstvu pod sebou.

vrstva	moduly	odpovědnost
shell	`shell`, `command`	čte řádky, parsuje je na `Command`, volá příkaz a vypisuje výsledek
příkazy	`command/*`	tenké funkce `do_*`: složí operace FS a převedou výsledek na `Output`
souborový systém	`filesystem`, `filesystem/*`	superblok, FAT, adresáře, překlad cest, check a bug
disk	`filesystem/disk`	čtení a zápis bajtů na daný offset souboru s FS

Mimo tyto vrstvy stojí `host` (soubory na hostitelském disku pro `incp` a `outcp`), `bytes` (převod čísel na bajty) a `config` (konstanty).

Struktura `FileSystem` drží otevřený disk, načtený superblok, celou FAT v paměti a cluster aktuálního adresáře. `Shell` drží `Option<FileSystem>`: `None` znamená nezformátovaný disk, kdy funguje jen `format` a ostatní příkazy skončí chybou `Disk is not formatted.`

2 Architektonická rozhodnutí

2.1 Návrh

2.1.1 Rozložení disku

offset	oblast	velikost
0	superblok	64 B
`fat_start`	FAT	$N \times 4$ B
`data_start`	clustery $0 \ldots N-1$ (cluster 0 je kořen `/`)	$N \times 1024$ B
	nevyužitý zbytek	méně než 1028 B

Počet clusterů je $N = \lfloor (\text{disk_size} - 64)/(1024 + 4) \rfloor$, protože každý cluster potřebuje 1024 B dat a 4 B ve FAT.

Superblok (ve skutečném FAT nazývaný „boot sector“) popisuje celý disk. Program ho čte jako první a vše ostatní zjistí z něj. Všechna čísla jsou little-endian.

offset	velikost	pole	příklad (600 MiB)	význam
0	8 B	`magic`	`PSEUDFAT`	poznání vlastního FS
8	4 B	`version`	1	pro budoucí změny formátu
12	4 B	`cluster_size`	1024	
16	4 B	`cluster_count`	612 009	N
20	4 B	`root_cluster`	0	cluster kořenového adresáře
24	8 B	`disk_size`	629 145 600	kontrola proti skutečné délce souboru
32	8 B	`fat_start`	64	začátek FAT
40	8 B	`data_start`	2 448 100	začátek clusteru 0
48	16 B	rezerva	nuly	

Při otevření se ověří `magic` (jinak „nezformátováno“), `version`, shoda `disk_size` s délkou souboru a nakonec se rozložení spočítá znovu ze `disk_size` a `cluster_size` a musí se shodovat s uloženým. Poškozený superblok se tak nepoužije.

2.1.2 FAT tabulka

FAT je pole `cluster_count` čísel `u32` hned za superblokem. Položka i říká, co následuje po clusteru i:

hodnota na disku	`FATPtrEntry`	význam
`u32::MAX`	`FREE`	cluster je volný
`u32::MAX - 1`	`EOF`	poslední cluster řetězce
`u32::MAX - 2`	`BAD`	vadný cluster (jen pro úplnost, viz níže)
cokoliv jiného	`Next(n)`	další cluster téhož souboru je n

- FAT ví jen to, že po clusteru 2 následuje cluster 3. Kde soubor začíná, ve FAT není: první cluster je uložený v položce adresáře.
- Skutečný FAT čísluje datové clustery od 2, protože hodnota `FREE` je tam 0. Kdyby data mohla ležet v clusteru 0, znamenalo by `FAT[i] = 0` buď „volno“, nebo „další je cluster 0“.

Tady jsou speciální hodnoty na konci rozsahu `u32`, takže se čísluje od 0. Z toho plyne i limit `MAX_CLUSTER_COUNT = u32::MAX - 2`.

- `BAD` je uvedená spíše pro úplnost. Zadání ji nevyžaduje a implementace ji plně nevyužívá: žádný cluster se jako vadný neoznačuje, jen `get_chain` odmítne řetězec, který na `BAD` narazí.
- Skutečný FAT drží dvě kopie tabulky jako zálohu. Tady je jen jedna.
- Celá FAT je v paměti a každá změna se hned zapíše i na disk (write-through). Soubor s FS je tak po každé operaci aktuální.

Proč 32bitové položky. FAT12 adresuje zhruba 4 080 clusterů, FAT16 zhruba 65 520, FAT32 (28 bitů z 32) zhruba 268 milionů. Disk 600 MiB s clustery po 1 KiB má $600 \times 1024 = 614\,400$ clusterů (po odečtení režie zhruba 612 tisíc). To je víc, než zvládne FAT16, takže pro malé 1 KiB clustery připadají v úvahu jen 32bitové položky. Jedna FAT pak zabere $614\,400 \times 4$ B $= 2\,457\,600$ B $\approx 2{,}34$ MiB. Velikost FAT závisí na velikosti clusteru:

cluster	**clusterů na 600 MiB**	**1 tabulka**	**2 tabulky**
512 B	1 228 800	4,69 MiB	9,38 MiB
1 KiB (použito)	614 400	2,34 MiB	4,69 MiB
4 KiB (běžně FAT32)	153 600	0,59 MiB	1,17 MiB

2.1.3 Limity velikostí

- `size` v položce adresáře je `u32`, jeden soubor má tedy nejvýš zhruba 4 GiB. `cluster_count` je také `u32`, ale omezuje celý disk (až $4{,}29 \cdot 10^9 \times 1024$ B $\approx 4{,}4$ TB). Jde o dvě nezávislé veličiny se stejným stropem.
- Počet clusterů souboru se neukládá, dopočítá se jako $\lceil s/c \rceil$, kde s je `size` a c `cluster_size`. Protože $c \geq 1$, platí $\lceil s/c \rceil \leq s$: v nejhorším (cluster 1 B) jsou si rovny, jinak je clusterů méně. Když se `size` vejde do `u32`, vejde se tam i počet clusterů souboru.
- `MAX_DISK_SIZE = 4 GiB` je čistě vývojářský a testovací limit. Hlavně brání tomu, aby překlep ve `format` při testování vytvořil obrovský soubor. Na správnost nemá vliv: přetečení počtu clusterů hlídá `MAX_CLUSTER_COUNT` při vytváření superbloku a velikost souboru převod na `u32` v `create_file`. Zadání požaduje 10–600 MB.
- `MIN_CLUSTER_SIZE = 512` B a podmínka mocniny dvou jsou pojistky spíš pro programátora, kdyby měnil `CLUSTER_SIZE`. Používá se `CLUSTER_SIZE = 1024` B.

2.1.4 Adresáře

Adresář zabírá vždy právě jeden cluster (ve FAT má `EOF`, velikost 0) a nedá se zvětšovat. Cluster je rozdělený na sloty po 32 B. Každý slot je buď volný (samé nuly), nebo obsahuje položku:

offset	**velikost**	**pole**	**význam**
0	12 B	`name`	8 B jméno + 3 B přípona + `\0`, kratší části doplněné nulami (zadání: 12 B, formát 8.3)
12	1 B	`kind`	0 = volný slot, 1 = soubor, 2 = adresář (bajty 13–15 jsou nuly)
16	4 B	`size`	velikost v bajtech; pro `info` a pro určení, kolik dat posledního clusteru je platných
20	4 B	`start`	položka FAT: číslo prvního clusteru, nebo `EOF` (prázdný soubor)
24	8 B	rezerva	

Kapacita adresáře je $1024/32 - 2 = 30$ položek. Sloty 0 a 1 jsou `.` (odkaz na sebe) a `..` (odkaz na rodiče). Skutečný FAT je v kořeni nemá. Tady je má i kořen a jeho `..` ukazuje na něj samotného, takže `cd ..` v kořeni zůstane v kořeni. Uložení `.` a `..` stojí 2 sloty z 32, ale `cd ..` je pak jen čtení položky, `pwd` jde složit procházením nahoru a není potřeba držet cestu v paměti jako zásobník.

2.1.5 Překlad cesty

Každý příkaz s cestou začíná stejně: z textu jako `../docs/./s1.txt` je potřeba najít položku.

1. Pokud cesta začíná `/`, začne se v kořeni, jinak v aktuálním adresáři.
2. Cesta se rozdělí podle `/`, prázdné části (`a//b`, `a/`) se ignorují.
3. Pro každou mezilehlou část: `.` znamená zůstat, `..` přejít do rodiče. Jakékoliv jiné jméno musí existovat a být adresář, jinak `PATH NOT FOUND`.

Funkce `walk` projde smyčkou jen mezilehlé části a vrátí rodičovský adresář a poslední část jako `Option<&str>`. Poslední část se odřízne, protože se na ni nesmí použít pravidlo „musí existovat a být adresář“: je to cíl operace a jeho pravidla určuje příkaz.

konec cesty	**co to je**	**`walk` vrátí jako poslední část**
`.../b`, `b` je adresář	existující adresář	`Some("b")`
`.../f.txt`, `f.txt` je soubor	existující soubor	`Some("f.txt")`
`.../new`, `new` neexistuje	nová položka (`mkdir`, `incp`)	`Some("new")`
`.../.` nebo `.../..`	navigace	`None` (vyřešeno ve smyčce)
`/` nebo prázdná cesta	kořen	`None`

`walk` nerozhoduje mezi prvními třemi případy. Volající zpracuje poslední část jedním ze dvou způsobů:

- **najdi existující** (`cat`, `rm`, `cd`, `ls`, `info`, zdroj u `cp`, `mv`, `outcp`): položka musí existovat a mít správný typ,
- **rozděl na rodiče a nové jméno** (`mkdir`, cíl u `cp`, `mv`, `incp`): rodič musí existovat (jinak `PATH NOT FOUND`), jméno v něm ještě nesmí být (jinak `EXIST`) a musí splňovat formát 8.3.

2.1.6 Pořadí zápisů a operace

Obecné pravidlo: při vytváření se zapíšou data, potom FAT a jako poslední položka adresáře. Při mazání se nejdřív odstraní položka a pak se uvolní FAT. Když program spadne uprostřed, nejhorší výsledek jsou obsazené clustery, na které nic neukazuje (ztracené clustery). To je jen ztráta místa. Při opačném pořadí by mohla zůstat položka ukazující na volné clustery, ty by pak přepsal jiný soubor a data dvou souborů by se smíchala.

`incp s1 s2` (`cp` funguje stejně, jen čte data z našeho FS):

1. Načte zdroj, jinak `FILE NOT FOUND`.
2. Rozdělí cíl na rodiče a jméno (`PATH NOT FOUND`, `EXIST`, neplatné jméno).
3. Ověří, že má rodič volný slot, jinak `Directory is full.`
4. Ověří, že je volných $\lceil \text{size}/1024 \rceil$ clusterů. Kontroluje se předem, aby nezůstal napůl zapsaný soubor.
5. Zapíše data do volných clusterů (poslední bývá zaplněný jen částečně).
6. Zřetězí je ve FAT: `FAT[c1] = c2`, ..., `FAT[ck] = EOF`.
7. Zapíše položku do rodiče.

`rm s1:` najde položku (musí to být soubor), odstraní ji z rodiče (vynuluje slot) a každý cluster řetězce nastaví na `FREE`. Data se při mazání nepřepisují, původní bajty v clusterech zůstanou, dokud je něco nepřepíše (na tom stojí nástroje pro obnovu smazaných souborů).

`mkdir a1:` rozdělí cestu na rodiče a jméno (stejné kontroly jako u `incp`), najde 1 volný cluster, nastaví `FAT[c] = EOF`, zapíše do clusteru `.` a `..` a nakonec položku do rodiče.

`rmdir a1:` najde adresář, který nesmí být kořen ani aktuální adresář (předek aktuálního adresáře neprojde, protože není prázdný). Kromě `.` a `..` musí být prázdný, jinak `NOT EMPTY`. Odstraní položku z rodiče a nastaví `FAT[c] = FREE`.

`mv s1 s2:` přesouvá se jen položka adresáře. Ze starého rodiče se odstraní a do nového se zapíše s novým jménem. Data ani FAT se nemění, takže přesun 500MB souboru trvá stejně dlouho jako přesun 1 B. Přejmenování je `mv` v rámci jednoho adresáře. U adresáře se navíc přepíše jeho `..` a je zakázaný přesun adresáře do sebe sama. `s2` je vždy úplná cílová cesta včetně jména, `mv a.txt dir` tedy při existujícím `dir` skončí `EXIST`.

2.1.7 check a bug

`check` projde strom adresářů od kořene a u každého souboru ověří neplatné číslo clusteru, cyklus v řetězci a nesoulad `size` s délkou řetězce. Každý problém vypíše na jeden řádek s cestou k souboru, bez problémů vypíše `OK`.

`bug s1 [size|cycle|invalid]` poškodí soubor `s1`:

druh	co udělá	kdy jde použít
`size`	změní velikost v položce o jeden cluster	vždy
`cycle`	poslední cluster nasměruje na první	soubor má aspoň 1 cluster a nepoškozený řetězec
`invalid`	poslední cluster nasměruje na číslo mimo rozsah	soubor má aspoň 1 cluster a nepoškozený řetězec

Bez druhého argumentu se druh vybere náhodně jen z druhů použitelných na daný soubor, takže opakované `bug s1` nikdy neselže kvůli nepoužitelné volbě. Náhoda bere nanosekundy systémového času, bez externí knihovny. `bug` lze volat opakovaně na různé soubory a `check` pak nahlásí všechna poškození.

Poškozený soubor nejde smazat: `rm` napřed ověřuje řetězec, takže soubor s cyklem nebo neplatným clusterem skončí chybou. Jediná cesta zpět je `format`.

2.1.8 load

Skript pro `load` musí ležet v našem FS, nejdřív se tam tedy musí zkopírovat přes `incp`. Bez zformátovaného FS tak nefunguje nic kromě `format`. Chyba jednoho příkazu nezastaví zbytek skriptu. `exit` ve skriptu ukončí celý program. Vnořený `load` je zakázaný, protože skript načítající sám sebe by nikdy neskončil.

2.1.9 MiB a MB

Zadání uvádí velikost v MB (`format 600MB`). Program jednotku chápe jako mocninu dvou, tj. MiB: `600MB` vytvoří disk 600×1024^2 B. Správně by se tedy mělo psát MiB, zadání to tak ale nepíše a vyžaduje zápis `MB`. Proto se v textu i programu používá `MB`, přestože jde o MiB.

2.1.10 Multiplatformnost

Program funguje na Windows i Linuxu:

- všechna čísla na disku jsou natvrdo little-endian, soubor s FS je tak stejný na každé platformě (nezávisí na pořadí bajtů procesoru),
- argumenty a cesty se čtou jako `OsString`/`PathBuf` (Linux UTF-8, Windows UTF-16),
- konce řádků LF i CRLF se zpracují stejně (vstup i skripty pro `load`),
- výzva `pseudofat>` se vypisuje jen při čtení z terminálu, ne z roury,
- `Makefile` rozlišuje Windows a ostatní systémy.

2.2 Implementace

2.2.1 Příkazy jako `do_*`

Každý příkaz je v samostatném souboru `command/<jméno>.rs` jako funkce `do_<jméno>` v bloku `impl FileSystem`. Výjimky jsou dvě. `format` je volná funkce, protože pracuje i bez otevřeného FS. `load` je v bloku `impl Shell`, protože na rozdíl od ostatních příkazů vykonává další příkazy a víc spolupracuje se shellem (viz 2.2.3). Logika, která potřebuje privátní pole `disk` a `fat`, je v `filesystem.rs` nebo jeho podmodulech (`paths.rs`, `consistency.rs`), protože podmodul vidí privátní pole rodiče. Příkazy pak jen skládají tyto operace.

2.2.2 Umělé `Next` v `get_chain`

`get_chain(first: FATPtrEntry)` nebere číslo clusteru, ale položku FAT. Pole `start` v položce adresáře je uložené jako `FATPtrEntry`: neprázdný soubor má `Next(první cluster)`, prázdný `EOF`. `get_chain` pak se startem zachází jako s „ukazatelem na první cluster“ a vrací *všechny* clustery včetně prvního. Kdyby brala přímo první cluster, první by se vynechal nebo by se musel řešit zvlášť. Prázdný soubor dá bez zvláštní větve prázdný řetězec. Proto i `DirEntry::dir` zabaluje cluster adresáře do `Next`.

```
let clusters = self.fat.get_chain(entry.start)?;
// start = Next(2) -> [2, 3, 4, 7, 10]
```

2.2.3 `pub(crate)` v `shell.rs`

Z funkcí, které vykonávají příkazy (`run`, `execute`, `process_line`), je `process_line` jediná `pub(crate)`. Spolu s ní jsou `pub(crate)` i `Flow`, `require_fs`, `is_load_running` a `set_load_running`. Důvod: každý příkaz má být v samostatném souboru, ale `load` jako jediný potřebuje vnitřnosti `Shell` (vykonat řádek a vypsat výsledek). Jeho `impl Shell` v `command/load.rs` leží mimo modul `shell`, takže tyto položky musí být viditelné v rámci crate. Je to vědomá nekonzistence ve viditelnosti výměnou za jednotnou strukturu příkazů.

2.2.4 Dlouhé názvy v `paths.rs`

Funkce v `paths.rs` jsou velmi specifické a mají dlouhé ale dobře čitelné názvy (`get_parent_dir_and_name_from_path`, `get_dir_cluster_from_path`, `get_node_from_path`, ...). Každá odpovídá jednomu způsobu zpracování poslední části cesty.

2.2.5 Další chytáky

- `get_node_from_path` vrací `Option`: `None` znamená kořen, který nemá rodiče ani položku se jménem. Adresář se hledá přes cluster (`dir_of`), ne podle jména z cesty, protože cesta může končit `.` nebo `..` a pak skutečné jméno neobsahuje.
- `read_file` odmítne soubor, jehož řetězec neodpovídá velikosti (jinak by vrátil nesmyslná data). `info` tuto kontrolu nedělá a nesoulad jen vypíše jako `MISMATCH`, aby šlo poškození po `bug` ukázat.
- V `move_entry` jsou při přejmenování ve stejném adresáři `parent` a `target` dvě kopie téhož clusteru. Použije se jen `replace` na jedné z nich (funguje i v plném adresáři). Zápis přes obě kopie by jednu změnu přepsal druhou.
- Smyčky, které stoupají přes `..` (`is_dir_inside`, `get_path_from_cluster`) a prochází řetězec (`get_chain`), mají strop `cluster_count` kroků. V poškozeném FS by jinak cyklus znamenal nekonečnou smyčku.

3 Největší problémy

Zabránění `load` uvnitř `load`. Skript, který načítá sám sebe, by běžel donekonečna. `Shell` má příznak `loading`, který se nastaví po dobu běhu skriptu. Vnořený `load` pak skončí chybou `nested load`.

Ukládání změn FAT (RAM a disk). FAT je celá v paměti kvůli rychlému hledání volných clusterů a procházení řetězců. Každá změna se ale hned zapíše i na disk (write-through), aby soubor s FS nikdy nezaostal za stavem v paměti. Zapisuje se jen změněný úsek tabulky, ne celá FAT.

Překlad cesty na cluster. Najít pro `/docs/images` cluster adresáře znamená projít části cesty od kořene nebo aktuálního adresáře, řešit `.` a `..` a poslední část nechat na volajícím (viz Překlad cesty). Hlavní potíž byla oddělit mezilehlé části (musí být adresáře) od poslední části (pravidla určuje příkaz).

Cesta ke clusteru (`pwd`). Adresář nezná své jméno, to je uložené v rodiči. Cesta se proto skládá odspodu: přes `..` se přejde do rodiče, v něm se najde položka, která ukazuje na výchozí cluster, a tak dál až ke kořeni. Jména se sbírají od nejhlubšího a na konci se otočí.

Efektivní zápis souboru. Clustery souboru se seskupí do souvislých úseků (extentů) a každý úsek se zapíše nebo přečte jedním voláním. Soubor v clusterech `[4, 5, 9]` se tak zapíše dvěma zápisy místo tří. Drobnost: změny FAT se ukládají jako jeden úsek od nejmenšího po největší dotčený cluster. Když má soubor jeden cluster na začátku a druhý na konci disku, přepíše se celá FAT.

Přesun adresáře. Při `mv` adresáře nestačí přesunout položku. Musí se přepsat jeho `..` na nového rodiče (jinak by `cd ..` a `pwd` vedly do starého umístění), zakázat přesun do vlastního podstromu (vznikla by smyčka odpojená od kořene) a všechny kontroly provést dřív, než se cokoliv zapíše.